

Clario Vision

Documentation Technique

v1.0 · 2025

Architecture · Modules Engine · Store · Composants UI · Export · Sécurité · Tests

Sommaire

01 Vue d'ensemble — architecture

02 Stack technique

03 Structure du projet

04 parseFec.js — Parsing FEC

05 exerciceUtils.js — Exercices

06 computeSig.js — SIG

07 computeTreasury.js — Trésorerie

08 computeBilan.js — Bilan

09 Modules complémentaires (1)

10 Modules complémentaires (2)

11 Store Zustand

12 Composants — Layout

13 Composants — Dashboard

14 Composants — Modules autonomes

15 Export PDF & PPTX

16 Auth · Sécurité · Tests · Deploy

Architecture — 100% client-side

Engine (logique métier pure)

parseFec · computeSig · computeTreasury · computeBilan · computeCharges · computeLivres

Store (Zustand)

State N · N-1 · N-2 · UI state · Actions async

UI (React 18 + Recharts)

Dashboard · Export PDF · Diaporama · Rapport IA · Livres

Aucun serveur. Aucune base de données. Le FEC est traité entièrement dans le navigateur de l'utilisateur.

Stack technique

Technologie	Rôle	Détail
React 18	Framework UI	Composants fonctionnels + hooks
Vite	Build tool	Dev server + bundler ESM
Zustand	State management	Store global centralisé, actions async
Recharts	Graphiques	BarChart, LineChart, AreaChart, PieChart
PapaParse	Parsing CSV	Web Worker intégré, encodages multiples
pdfmake	Export PDF	Génération 100% client-side, SVG embarqués
pptxgenjs	Export PPTX	Diaporama builder, layouts personnalisés
Tailwind CSS	Styling	Mode clair uniquement, design system FV
date-fns	Dates	Manipulation exercices comptables
Vitest	Tests	Unitaires + React Testing Library

Structure du projet

```
src/  
  engine/          ← Logique métier pure  
    parseFec.js  
    computeSig.js  
    computeTreasury.js  
    computeBilan.js  
    computeCharges.js  
    computeLivres.js  
    drillDown.js | formatUtils.js  
  components/      ← Composants React  
    layout/ sig/ monthly/ treasury/  
    charges/ balance/ export/ ...
```

```
store/  
  useStore.js      ← Zustand central  
  useAuthStore.js  
  useBilanParamStore.js  
  __tests__/       ← Tests Vitest  
    parseFec.test.js  
    computeSig.test.js ...  
main.jsx | App.jsx  
  
public/  
  demo/  
    demo_fec.csv | demo_fec_n1.csv  
    demo_bilan_cr.json ...
```

Entrée du pipeline de données

Détection automatique

Encodage : UTF-8, ISO-8859-15, Windows-1252 · Séparateur : pipe (|) ou tabulation

Validation header

18 colonnes réglementaires obligatoires dans l'ordre exact (JournalCode ... ldevise)

Web Worker

Parsing non-bloquant via PapaParse worker · Callback de progression (0–100%)

Objet de sortie

entries[] · siren · closingDate · encoding · separator · warnings[]

 **Règle stricte : 1 seul fichier FEC accepté — si multiple, erreur bloquante immédiate.**

Fonctions exportées

```
extractSiren(fileName)
  → { siren, closingDate }

detectExerciceStart(entries)
  → Date de début d'exercice

buildExerciceMonths(start, end)
  → [{ month, year, label }]

getMonthLabel(n)
  → 'Janvier', 'Février' ...

getExerciceLabel(start, end)
  → 'Exercice 2024'
```

Exercice décalé — exemple

avril 2024 → mars 2025

Séquence des mois :

[4, 5, 6, 7, 8, 9, 10, 11, 12, 1, 2, 3]

Pourquoi c'est important

Les graphiques (mensuels, trésorerie, comparaison) utilisent cette séquence pour respecter l'ordre chronologique de l'exercice, quelle que soit l'année civile.

Structure SIG

CA (701*–708*)

+ Subventions (74*) – Achats (60*)

= **Marge brute**

– Services ext. (61*, 62*)

= **Valeur Ajoutée**

– Personnel (64*, 621*) – Taxes

= **EBE**

± Dotations, Reprises, Autres

= **Résultat d'Exploitation**

± Financier ± Exceptionnel – IS

= **RÉSULTAT NET**

Spécificités CUMA

621* — Intérimaires

Classé en Personnel, PAS en Services ext. Décision métier CUMA validée.

706*–708* — Travaux agricoles

Comptes CUMA inclus dans le CA (prestations, travaux, services).

Journal ANC — Exclusion SIG

Les écritures d'à-nouveau ne contribuent PAS au SIG (bilan uniquement).

Données mensuelles

Calculées dans l'ordre de l'exercice (exerciceUtils) pour les graphiques.

Comptes suivis

51* Banques
53* Caisse

Solde d'ouverture

Journal ANC sur 51*/53*
(seule exception : ANC requis)

Convention comptable

Débit 51* = encaissement
Crédit 51* = décaissement

Calculs produits

Solde quotidien · Mini/Maxi/Moyen
Top 10 encaissements & décaissements

Structure

ACTIF

Immobilisé → 20*–27* (machines, installations)

Circulant → 3*, 4* (stocks, créances)

Trésorerie → 50*–53* (banques, caisse)

PASSIF

Capitaux propres → 10*–13*

Dettes MLT → 16* (emprunts)

Dettes CT → 40*, 42*–47*

4 ratios : FR · BFR · Autonomie financière · Liquidité générale

Spécificité CUMA — Compte 453*

Adhérents — travaux & cessions

1. Calculer le solde NET global de tous les 453*
2. Si solde débiteur → ACTIF circulant
→ 'Créances adhérents nettes'
3. Si solde créditeur → PASSIF dettes
→ 'Dettes adhérents'
4. Le drill-down montre le détail
par compte auxiliaire (par adhérent)

Modules engine — computeCharges.js & drillDown.js

computeCharges.js

6 catégories CUMA : Personnel (64*, 621*) · Services ext. (61*, 62* –621*) · Dotations (68*) · Achats (60*) · Financières (66*) · Impôts & taxes (63*)

Répartition % par catégorie · Données mensuelles par catégorie · 621* exclu des services ext. (reclassé en personnel)

drillDown.js

Niveau 1 : Pour un poste SIG ou bilan → liste des comptes contribuant (nb écritures, total débit/crédit, solde)

Niveau 2 : Pour un compte donné → liste des écritures triées par date, avec solde running cumulé

Partagé entre les onglets SIG, Bilan et Balance · Composant DetailPanel commun

Modules engine — Livres comptables & Utilitaires

computeLivres.js

- Balance générale — solde par compte (débit, crédit, solde net)
- Balance auxiliaire — détail par compte auxiliaire (fournisseurs, clients...)
- Grand Livre général — toutes les écritures détaillées, triées par compte

formatUtils.js

- formatAmount(n) → '18 750 €' ou '482 k€' (Intl.NumberFormat fr-FR)
- formatPercent(v) → '33,0 %'
- formatDate(d) → '10/01/2024'

parseDossierGestion.js + parseBilanCR.js

- Parsing des sources externes : Dossier de gestion (JSON) + Bilan CR Divalto (JSON)
- Validation de structure + normalisation des montants vers les types attendus par le store

Store Zustand — useStore.js

État exercice N

`parsedFec`

`sigResult`

`treasuryData`

`chargesData`

`bilanData`

`analytiqueData`

`dossierData`

`bilanCRData`

`analyseIAText`

État N-1 & N-2

`parsedFecN1 / N2`

`sigResultN1 / N2`

`treasuryDataN1`

`chargesDataN1`

`bilanDataN1 / N2`

`isLoadingN1 / N2`

`errorN1 / N2`

État UI

`activeSection`

`activeTab / SubTab`

`detailPanel`

`isLoading`

`loadProgress`

`error / warnings`

`diaporamaSlides`

`diaporamaCover`

Composants — Structure Layout

AppHeader — Logo · Nom du fichier FEC · Période d'exercice · Actions (démon, reset)

KpiBar — CA · EBE · Résultat net · Solde trésorerie (avec Sparklines)

TabNav — SIG · Analyses · Trésorerie · Charges · Bilan · Comparaison · Analytique

Zone de contenu actif
(composant du tab sélectionné)

SideNav

11 sections

Accueil
Analyseur
Dashboard
Dossier
Bilan CR
Bilan param.
Éditions
Export
Diaporama
IA
Admin

Composants partagés : Badge · Sparkline · Tooltip · ErrorBanner · SubTabNav

Composants — Dashboard (7 onglets)

Onglet	Composants principaux
SIG	SigTable · SigRow · DetailPanel · AccountList · AccountCard · EntryTable · EntryFilter
Analyses mensuelles	MonthlyTab · MonthlyCharts · CumulativeChart · MonthlyDataTable
Trésorerie	TreasuryTab · TreasuryKpis · TreasuryCurve · TopMovements · PeriodToggle
Structure des charges	ChargesTab · ChargesDonut · ChargesDetailList · ChargesMonthlyChart
Bilan simplifié	BalanceTab · BalanceOverview · RatioCards · AssetSection · LiabilitySection · BalanceRow
Comparaison N/N-1	ComparaisonTab — tableaux annuels + graphiques SVG + tableaux mensuels
Analytique	AnalytiqueTab — répartition par matériel, podium top 3

Composants — Modules autonomes

Analyseur FEC

Score qualité
Anomalies & balance journal

Dossier de gestion

Synthèse, Résultats,
Charges, Financement

Bilan & CR Divalto

Actif / Passif / Résultat
(source Divalto)

Bilan paramétré

Structure configurable
useBilanParamStore

Livres comptables

Balance · Balance aux.
Grand Livre

Export PDF

13 types de docs
generatePdf.js

Diaporama

Builder de slides
generatePptx.js

Rapport IA

Streaming Anthropic
Markdown → JSX

Administration

Panneau admin
(useAuthStore)

13 types de documents

- Dossier de gestion
- Bilan simplifié
- Balance générale
- Grand Livre
- Structure des charges
- Analytique (podium)
- Comparaison N/N-1
- SIG
- Bilan & CR Divalto
- Balance auxiliaire
- Trésorerie (courbe)
- Analytique (table)
- Rapport IA

Fonctionnalités

Cover page dynamique

Logo upload · Sommaire auto · SIREN · Période

SVG graphiques

Construits en JS pur (pas de capture DOM)
Trésorerie, charges, comparaison N/N-1

Unbreakable blocks

Graphique + tableau jamais séparés
par un saut de page

Portrait / Paysage

Colonnes 'auto' → adaptation automatique
à la largeur de page

Séquence libre

L'utilisateur choisit et réordonne
les documents dans l'export

Export PPTX — generatePptx.js & Diaporama builder

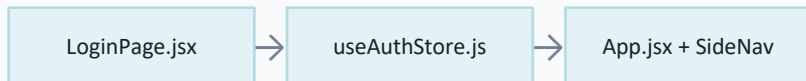
generatePptx.js

- Génération PowerPoint 100% client-side
- Utilise pptxgenjs (même librairie que ce document !)
- htmlToPptxRuns.js : conversion du rich text HTML → runs PPTX formatés
- diaporamaConfig.js : définition des layouts disponibles (positions, zones)

Diaporama builder (DiaporamaTab)

- Constructeur visuel de slides
- Zones éditables — RichTextEditor.jsx (texte formaté, listes, titres)
- Sélection de layouts — LayoutPicker.jsx (dispositions prédéfinies)
- Liste des slides réordonnables — SlideList.jsx
- Export final vers .pptx via generatePptx.js

Authentification



AdminPanel accessible aux seuls admins (flag isAdmin dans le store)
Session conservée en mémoire uniquement — pas de localStorage

Sécurité — données

-  100% client-side — aucune transmission réseau du FEC
-  Web Worker terminé + URL révoquée après parsing
-  Pas de localStorage ni sessionStorage
-  Build statique — CSP stricte compatible

Vos données FEC ne quittent jamais votre navigateur.

Aucune clé API côté client exposée dans le bundle · Aucun compte utilisateur requis · Conforme RGPD par conception

Tests — Vitest + React Testing Library

Fichier de test	Ce qui est testé
parseFec.test.js	3316 lignes parsées · encodage ISO-8859-15 · séparateur · SIREN 381304559 · montants FR → float
computeSig.test.js	Totaux SIG cohérents · exclusion journal ANC · cohérence sous-totaux → lignes de total
computeTreasury.test.js	Solde d'ouverture ANC · solde quotidien · top 10 encaissements/décaissements
computeBilan.test.js	Équilibre actif = passif · classification 453* (débit → actif, crédit → passif) · ratios
computeLivres.test.js	Balance générale · balance auxiliaire · grand livre — cohérence des soldes
exerciceUtils.test.js	Exercice décalé [4..3] · extractSiren · getExerciceLabel · getMonthLabel

```
$ npm run test
```

Déploiement

1 Build

npm run build
→ dossier dist/ (assets statiques)

2 Hébergement

Copier dist/ sur Nginx / Apache /
GitHub Pages / Vercel / Netlify

3 SPA routing

Ajouter un fichier _redirects ou une
règle rewrite → toutes routes →
index.html

Variables d'environnement Vite

Variable	Description
VITE_API_URL	URL de l'API (Anthropic) pour le Rapport IA
VITE_AUTH_ENABLED	Activer / désactiver l'authentification (true/false)

Application statique — aucun serveur Node.js requis en production.

Clario Vision

Documentation Technique — fin

Architecture · Engine · Store · UI · Export · Sécurité · Tests · Déploiement

v1.0 · 2025